

Web Scraping with BeautifulSoup

Learn how to automate web data collection using Python's powerful libraries: Requests & BeautifulSoup.

1. Introduction: Manual vs. Automated Data Collection

Imagine you are tracking live scores for your favorite sports tournament, such as IPL or ISL. Copying and pasting scores manually line-by-line is slow, tedious, and prone to human error. But can a program do this task for you?

Manual Collection

Copy-pasting every single score or piece of text by hand takes hours and requires constant monitoring.

Automated Scraping

A smart Python script collects, structures, and saves all data automatically in seconds!

2. What Is Web Scraping?

Web scraping means writing a computer program that automatically reads web pages and extracts the specific information you need.

Analogy: Think of web scraping like a *Dabbawala* or a food delivery driver who picks up lunchboxes from different locations and delivers them straight to your home. Similarly, a scraping script extracts scattered data across web pages and brings it directly to your program! Think of your program as a web detective searching for clues.

3. Spotting HTML Tags

Web pages are built using **HTML** (HyperText Markup Language). To extract data, your program must identify and read specific HTML tags:

HTML Tag	Meaning / Description	Example Use Case
<h1>	Big Heading	Main title of an article or webpage
	Unordered List	A bulleted list container
	List Item	An individual item inside a list

4. Core Tools: Requests & BeautifulSoup

Web scraping in Python relies on a team effort between two key tools:

- **Requests:** Acts like the postman bringing the HTML webpage code from the internet straight to your Python program.
- **BeautifulSoup:** Acts like the detective parsing (reading) the raw HTML markup and finding specific elements or content.

5. Code Example: Fetching National Parks Data

Here is a live coding snippet demonstrating how to fetch and parse data:

```
import requests
from bs4 import BeautifulSoup

# Step 1: Request page contents from web server
url = "https://example.com/india-national-parks"
response = requests.get(url)

# Step 2: Parse HTML content with BeautifulSoup
soup = BeautifulSoup(response.text, "html.parser")

# Step 3: Extract main headline and top 5 national parks
headline = soup.find("h1").text
parks = soup.find_all("li")[:5]

print("Headline:", headline)
for park in parks:
    print("-", park.text)
```

6. Finding Elements: `find()` vs. `find_all()`

Understanding the difference between getting *one* item versus *many* items is crucial:

Method	Behavior	Analogy	Code Example
<code>find()</code>	Returns the FIRST matching element.	Picking the first mango from a basket	<code>soup.find("h1")</code>
<code>find_all()</code>	Returns a list of ALL matching elements.	Collecting ALL mangoes at once	<code>soup.find_all("li")</code>

7. Ethical Web Scraping Rules

Always be a polite and responsible programmer when collecting data from websites:

- ✓ **Check Permissions:** Inspect the site's `robots.txt` file to see if web scraping is allowed.
- ✓ **Don't Overload Servers:** Avoid sending hundreds of requests per second to keep the website running smoothly.

- ✓ **Use Data Responsibly:** Extract and utilize data only for legal and constructive purposes.
- ✓ **Respect Privacy:** Never attempt to scrape private or confidential user information.

Practice Questions

Test your understanding of Python web scraping and BeautifulSoup. Complete all questions below.

Question 1: Conceptual Knowledge

Explain the difference between manual data collection and automated web scraping. Why is web scraping preferred for large datasets?

Question 2: Tool Roles

Describe the specific roles of the `requests` library and the `BeautifulSoup` library in a Python web scraping script.

Question 3: HTML Tag Matching

Match the following HTML tags to their correct description:

- a) `<h1>`
- b) `<ul>`
- c) `<li>`

Question 4: BeautifulSoup Methods

Consider a webpage containing multiple paragraph (`<p>`) tags:

1. What method would you use if you only want to retrieve the very first paragraph?
2. What method would you use if you want to extract every paragraph on the page into a list?

Question 5: Code Analysis & Output Prediction

Given the following Python code snippet, write down what output will be printed assuming the HTML page contains three list items ("Apple", "Banana", "Cherry"):

```
import requests
from bs4 import BeautifulSoup

html_data = "<ul><li>Apple</li><li>Banana</li><li>Cherry</li></ul>"
soup = BeautifulSoup(html_data, "html.parser")
items = soup.find_all("li")
```

```
for item in items:  
    print(item.text)
```

Question 6: Ethics in Scraping

List three ethical rules or best practices you must follow before and while web scraping a live website.